

CI/CD PIPELINE ASSIGNMENT

Flask and MongoDB App on EC2

Docker, Amazon ECR, EC2. Jenkins or GitHub Actions.

[Reference guide for learners](#)

What You Are Building

A working CI/CD pipeline that takes the Student Registration System (Flask and MongoDB) from a git push all the way to a running container on EC2. Fully automated, tested, and self reporting by email.

Repository

github.com/mohanDevOps-arch/flask_Practice

Fork this repo. It is your starting point for the assignment.

Tech stack

Python

Flask

MongoDB

Docker

AWS (ECR + EC2)

You will add a Dockerfile, a /health endpoint, and a pipeline file. The app logic itself does not need to change.

What You Need To Do

1 Fork and clone

Fork the flask_Practice repo into your own GitHub account and clone it locally.

2 Add a Dockerfile

Write a Dockerfile that builds and runs the Flask app on port 5000.

3 Add a /health route

Add a route that checks MongoDB connectivity. This is your deploy verification gate.

4 Provision AWS

Create an ECR repo and an EC2 instance (Docker installed, IAM role for ECR pull, security group rules for ports 22 and 5000).

5 Write the pipeline

Create a Jenkinsfile OR a GitHub Actions workflow, not both. Pick one tool for this assignment.

6 Configure email alerts

Set up success and failure email notifications with a customized message for each outcome.

7 Test it end to end

Push a change, confirm the pipeline runs and deploys, and check that you receive the correct email.

8 Document and submit

Update the README, then submit your repo link through Vlearn.

Required Pipeline Stages

Same stages whether you build this in Jenkins or GitHub Actions.

1

Checkout

Pull latest source on push to main.

2

Install

pip install from requirements.txt.

3

Test

Run pytest. Stop here if any test fails.

4

Build

Build a Docker image tagged with the commit SHA.

5

Push to ECR

Authenticate and push the tagged image.

6

Deploy to EC2

SSH in, pull the image, and replace the running container.

7

Verify

Poll /health. This is the real success gate.

8

Notify

Send an email with a customized success or failure message.

Required Folder Structure

```
flask_Practice/
├── app.py
├── requirements.txt
├── test_app.py
├── templates/
│   ├── base.html
│   ├── index.html
│   ├── add_student.html
│   └── update_student.html
├── Dockerfile                (you add this)
├── .dockerignore             (you add this)
├── Jenkinsfile               (OR this)
├── .github/workflows/
│   └── ci-cd.yml              (OR this)
├── .env.example              (you add this)
└── README.md                 (you update this)
```

Never commit

The .env file, real Mongo URIs, .pem keys, or AWS access keys. Use .env.example as the template and add the rest to .gitignore.

One pipeline file

Add either a Jenkinsfile or .github/workflows/ci-cd.yml, not both. Pick one tool for this assignment.

Keep app.py logic untouched

Only add the /health route. Do not restructure the existing routes or templates.

Do's

- ✓ Tag every Docker image with the git commit SHA, not just latest.
- ✓ Store every secret (Mongo URI, SSH key, AWS credentials, SMTP credentials) in Jenkins Credentials or GitHub Secrets.
- ✓ Make the pipeline fail the build if any pytest test fails. Do not let a broken app reach EC2.
- ✓ Verify the deployment with a real health check before declaring success.
- ✓ Write a customized email body. Include the commit SHA, the image tag, and on failure, which stage broke.
- ✓ Test your pipeline with an intentionally failing commit at least once, so you know the failure path actually works.
- ✓ Commit your Dockerfile, pipeline file, and updated README together, early.

Don'ts

 Do not hardcode the Mongo URI, AWS keys, or SSH private key anywhere in the repo.

 Do not skip the Test stage or comment it out just to get the pipeline green.

 Do not use docker run without --restart unless-stopped. A reboot should not kill your app.

 Do not leave port 22 or 5000 open to 0.0.0.0/0 without knowing why.
Document your security group choice.

 Do not send a generic "build finished" email. Success and failure must look different.

 Do not submit a repo where the pipeline has never actually run successfully.

 Do not wait until the last day to ask for help. See the last slide.

Grading Policy

Pipeline stages present and passing in order	25%
Docker image built and tagged with commit SHA	15%
Image successfully pushed to ECR	10%
EC2 deployment replaces the running container correctly	20%
Health check verification genuinely gates success or failure	10%
Email notification: customized content, both outcomes	15%
README documentation quality	5%

Pass or fail gate

A pipeline that has never had one fully successful run, including the deploy step, will not be graded above 40 percent, regardless of code quality.

Submission Checklist

GitHub repo link (forked flask_Practice, with your Dockerfile and pipeline file committed)

Pipeline file: Jenkinsfile or .github/workflows/ci-cd.yml

Updated README.md documenting setup, secrets, and how the deploy step works

Screenshot or short recording of a full successful pipeline run

Screenshot of the success email received

Screenshot of a failed run (intentional) and the failure email received

Repo link submitted as a text, Word, or PDF file through Vlearn

Don't Wait For The Last Day

If you get stuck on AWS setup, SSH access, or the pipeline itself, reach out early. Deployment issues such as security groups, IAM roles, and SMTP setup often take a day or two to sort out.

support@herovired.com

or message us directly on the batch WhatsApp group

We would rather answer five small questions on Tuesday than one big one at midnight before the deadline.